Appendix A. Retrieval Benchmark Comparison

This appendix compares the three vector-store options used in Track B: Chroma, FAISS, and Qdrant.

The benchmark corpus is generated at four sizes: 100, 1,000, 10,000 and 50,000 documents. FAISS and Chroma run in-process; Qdrant is the Docker server, measured over gRPC and REST. The measurements below come from the run recorded in results.csv; the method is in experiment_protocol.md.

A.1 Quantitative Benchmark Matrix

Store (deployment)	Corpus	Index build (s)	Unfiltered P50 / P95 (ms)	`patient_id` -filtered P50 / P95 (ms)
FAISS flat (in-process)	100	0.0005	0.08 / 0.10	0.04 / 0.04
Chroma (in-process)	100	0.32	1.19 / 1.65	1.02 / 1.55
Qdrant (server, gRPC)	100	0.82	2.33 / 3.08	2.58 / 3.82
Qdrant (server, REST)	100	0.85	15.98 / 31.05	16.38 / 31.24
FAISS flat (in-process)	1,000	0.002	0.52 / 0.58	0.05 / 0.05
Chroma (in-process)	1,000	0.57	1.05 / 1.53	1.95 / 2.86
Qdrant (server, gRPC)	1,000	0.96	2.59 / 3.07	2.69 / 3.15
Qdrant (server, REST)	1,000	0.98	15.96 / 31.16	16.61 / 28.80
FAISS flat (in-process)	10,000	0.02	5.18 / 5.89	0.13 / 0.15
Chroma (in-process)	10,000	3.30	1.77 / 2.24	13.13 / 17.52
Qdrant (server, gRPC)	10,000	2.87	2.93 / 3.73	2.48 / 3.21
Qdrant (server, REST)	10,000	2.95	19.21 / 32.38	17.13 / 30.84
FAISS flat (in-process)	50,000	0.05	3.50 / 5.48	0.26 / 0.28
Chroma (in-process)	50,000	16.70	1.62 / 2.18	61.46 / 71.24
Qdrant (server, gRPC)	50,000	11.43	5.28 / 6.44	2.57 / 3.20
Qdrant (server, REST)	50,000	11.46	16.15 / 35.35	15.91 / 32.35

Retrieval quality (FAISS and Qdrant searched exactly and are identical; Chroma differs where noted):

Corpus	Workload	Recall@3	Cross-patient leakage	Chroma ANN recall vs exact
100	unfiltered	0.26	0.93	1.00
1,000	unfiltered	0.21	0.96	1.00
10,000	unfiltered	0.21 (Chroma 0.15)	0.93	0.93

Corpus	Workload	Recall@3	Cross-patient leakage	Chroma ANN recall vs exact
50,000	unfiltered	0.19 (Chroma 0.06)	0.97	0.73
all sizes	`patient_id` filter	1.00	0.00	1.00

A.2 Qualitative Comparison

Store	Install Effort	Persists to Disk	Metadata Filtering	Practical Notes
Chroma	Low to moderate	Yes, with a persistent client and persistence directory	Yes	Easy to use for local prototyping and supports metadata-based patient lookup.
FAISS	Low	Not by default as a full vector database	No native metadata filtering	Fastest raw search path, but you need a separate metadata store for patient_id filtering and persistence.
Qdrant	Moderate	Yes, when run with storage enabled	Yes	Strong choice when you need both vector search and payload filtering in one system.

A.3 Interpretation

- **Without a filter, patient retrieval does not work.** For a query that names a patient, 93-97% of the returned documents belong to a different patient, in every store. The embedding model cannot tell `P00412` from `P00421`; unfiltered Recall@3 of about 0.2 is carried almost entirely by the seven general-knowledge queries. Filtering is a correctness requirement here, not a convenience.
- **With the filter, Qdrant's latency does not grow with the corpus**: about 2.5 ms median over gRPC from 100 to 50,000 documents. Chroma's filtered latency grows from 1.0 ms to 61 ms over the same range. FAISS is the fastest (0.04-0.26 ms), but only because the benchmark keeps its own `patient_id -> row ids` map beside the index, which is exactly the metadata layer FAISS does not provide.
- **The transport matters more than the engine at this scale.** The same Qdrant server costs about 2.5 ms per query over gRPC and about 16 ms over REST on the benchmark machine (Docker Desktop on Windows). `retrieval_service.py` currently connects over REST.
- **FAISS is not uniformly the fastest.** The exact flat index is fastest up to 1,000 documents; from 10,000 documents Chroma's HNSW index answers unfiltered queries faster (1.6-1.8 ms against 3.5-5.2 ms), at the price of approximate results: its top 3 agree with exact search 93% of the time at 10,000 documents and 73% at 50,000 with default settings.
- **Qdrant answered every size by exact scan.** With server defaults the collection is split into 8 segments, each below the indexing threshold, so no HNSW index was built even at 50,000 documents (`indexed_vectors_count` stayed 0). Its ANN recall of 1.00 follows from that, and its unfiltered latency grows with the corpus (2.3 ms to 5.3 ms).
- Filtered Recall@3 of 1.00 and leakage of 0.00 are what a correct filter must produce when a patient has two documents and `top_k = 3`. They verify the filter; they are not a retrieval-quality result.

The corpus is synthetic and templated, the three stores are not deployed the same way (two in-process libraries against one networked server), and one machine was used. The numbers support the design decision; they are not a general ranking of the three products. Method and reproduction steps: experiment_protocol.md; machine and versions: benchmark_environment.json.

If the goal is pure in-process speed on a small corpus, FAISS wins. If the goal is patient-scoped retrieval behind a persistent service, Qdrant over gRPC is the most balanced option. Chroma is a reasonable choice for rapid development and demonstrations at small scale.

Track B Vector-Store Benchmark Protocol

Purpose

This protocol defines the canonical process for comparing FAISS, Chroma, and Qdrant. It makes the benchmark outputs in `results.csv` and `raw_latencies/` reproducible and prevents figures, tables, and reports from being created from different runs.

What is compared

Store	Deployment in the benchmark	Index	Patient filter mechanism
FAISS	in-process library	`IndexFlatIP` (exact)	application-side: a `patient_id -> row ids` map turned into an `IDSelectorBatch` per query
Chroma	in-process, ephemeral client	HNSW, cosine	native metadata `where` clause
Qdrant	**server** in Docker (the Compose service), over gRPC	server default (HNSW once the optimizer indexes the segment)	native payload filter on a keyword-indexed `patient_id` field
Qdrant-REST	same server, over REST	same	same

The three deployments are not equivalent, and the report must say so wherever latencies are compared: FAISS and Chroma answer inside the benchmark process, while every Qdrant query crosses the network boundary into the container. The Qdrant rows are the latency the retrieval service would actually see; the in-process rows are a lower bound that excludes any service boundary. qdrant-client's `:memory:` mode is not used anywhere, because it is a pure-Python test stand-in, not the Qdrant engine.

Corpus and queries

Both are generated deterministically by `TrackB/retrieval/kb/generate_benchmark_corpora.py` (seeded by corpus size; nothing is read from or written to disk except the seven general notes in `kb/medical_notes.json`).

- Corpus sizes: 100, 1,000, 10,000 and 50,000 documents.
- Each corpus holds the 7 general clinical notes (no `patient_id`) plus one profile and one medical note per synthetic patient.
- Query set per corpus: 7 general-knowledge queries plus 2 queries (profile, note) for each of up to 20 sampled patients. Every query carries the ids of the documents that answer it.
- The data is synthetic and templated. Patient documents differ mostly by patient id, age, and a few rotating phrases, which makes cross-patient confusion easy to provoke. That is deliberate for the isolation measurement, and it means the recall numbers describe this corpus, not clinical text in general.

Workloads

Two workloads are timed for every store and corpus size:

- **unfiltered** — all labelled queries, searched across the whole corpus;
- **filtered** — the patient queries only, restricted to that patient's documents with the store's own filter mechanism.

Controlled variables

- Embedding model: `all-MiniLM-L6-v2`, normalized vectors, cosine / inner product.
- Search depth: `top_k = 3`.
- Timed samples: the query set is cycled in a seeded shuffled order until at least 300 searches were timed per store, corpus size and workload.
- Warm-up: 10 untimed searches before timing.
- Only the search call is timed. Query vectors are embedded beforehand, so the embedding cost (identical for all stores) is excluded.
- Index creation time includes loading the vectors and, for Qdrant, waiting until the collection status is green (optimizer finished).
- Each store runs in its own subprocess.

Metrics

- Index creation time (s).
- Query latency (ms): mean, standard deviation, min, P50, P95, P99, max, and the sample count.
- `recall_at_k`: share of labelled relevant documents found in the top 3.
- `cross_patient_leakage`: for patient queries, the share of returned documents that belong to a *different* patient.
- `ann_recall_vs_exact`: overlap between the store's top 3 and a brute-force cosine top 3 computed under the same filter. This separates "the index is approximate" from "the embedding ranks the wrong document first".

In the filtered workload each patient has at most two documents and `top_k = 3`, so filtered recall of 1.0 and leakage of 0.0 are what a correct filter must produce. Those two numbers verify the filter; they are not a retrieval-quality achievement.

Reproduction procedure

1. Start the Qdrant server:

   ```
   docker compose up -d qdrant
   ```

2. Run a fresh benchmark. `--fresh` removes prior aggregate and raw benchmark artifacts so the CSV matches the newly generated samples. The driver also writes `benchmark_environment.json` (machine, Python, package and Qdrant server versions, Git commit):

   ```
   python TrackB/retrieval/run_all_benchmark.py --fresh
   ```

3. Generate figures from exactly that aggregate CSV and raw samples:

```
python TrackB/retrieval/plot_benchmark.py
```

4. Verify that every `(store, documents_indexed, mode)` row has at least 300 samples and that all six figures were produced.

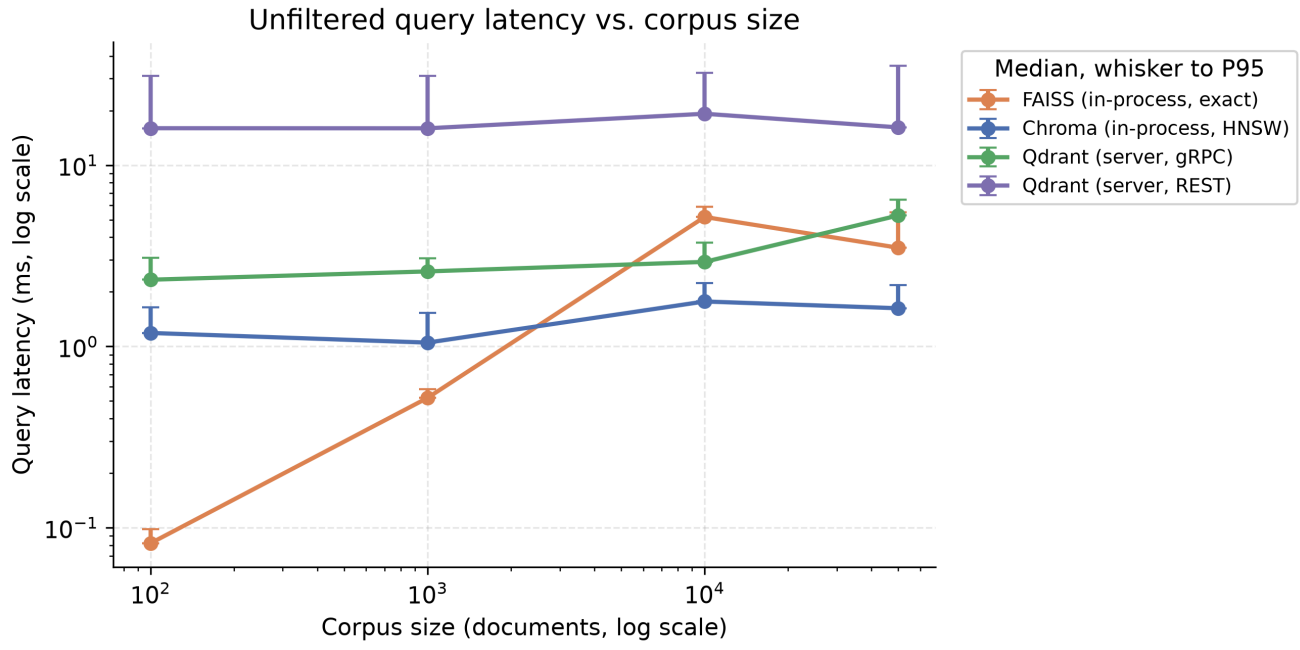
5. Update `FINAL_REPORT.md`, `README.md`, and `appendix_A.md` only from the canonical `results.csv`.

Close other CPU-heavy programs while the benchmark runs; latencies in the tens of microseconds are sensitive to background load.

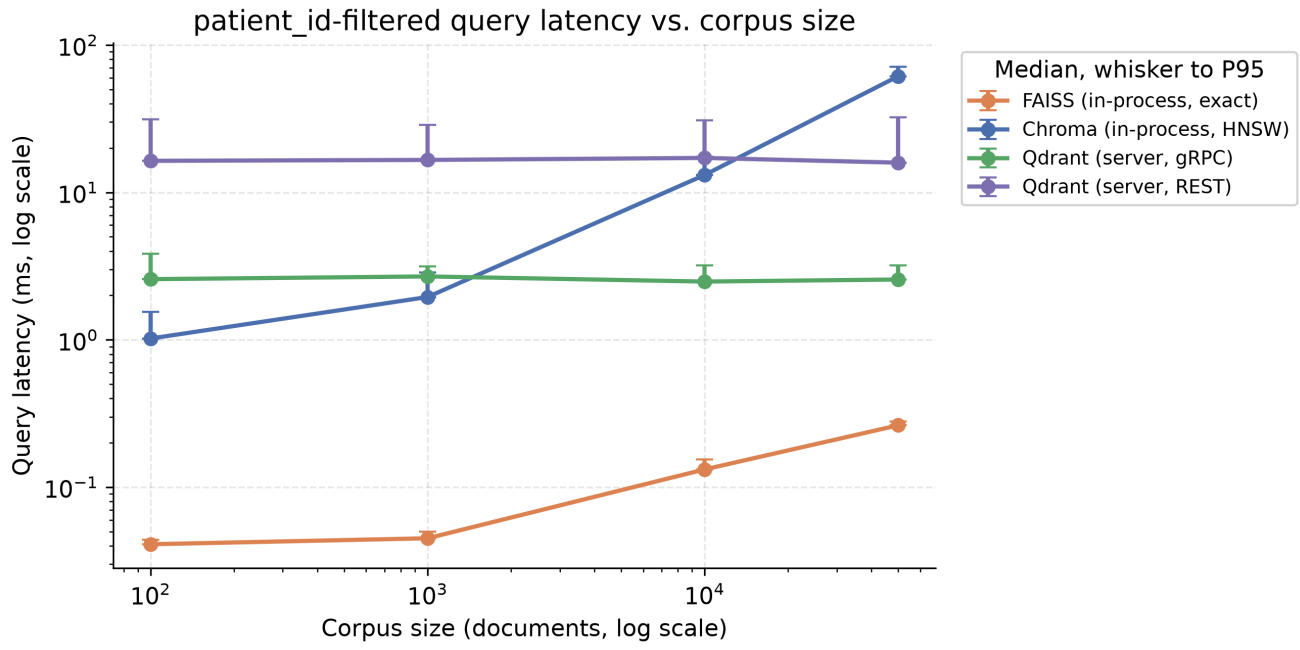
Reporting rules

- Report milliseconds for query latency and seconds for index creation time.
- Preserve all raw latency samples; never report a percentile without its underlying sample distribution.
- Do not combine rows from separate hardware, dependency versions, or runs in one comparison.
- Always state the deployment (in-process or server) next to a latency.
- A metric that could not be measured is reported as missing. It is never filled in from another run, another tool, or by hand.

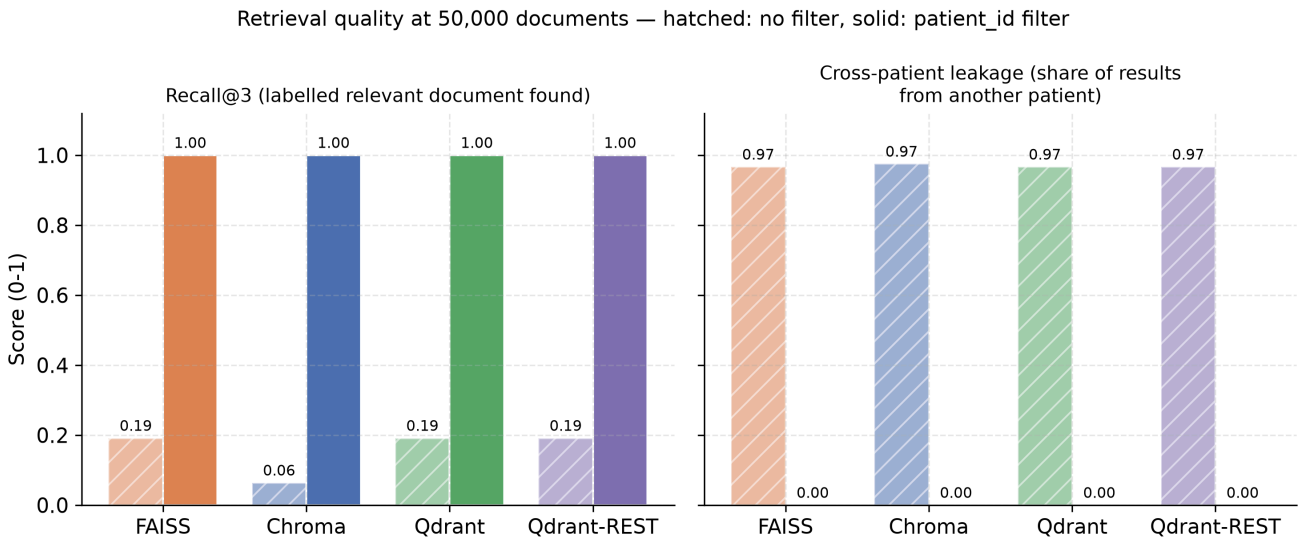
Figures



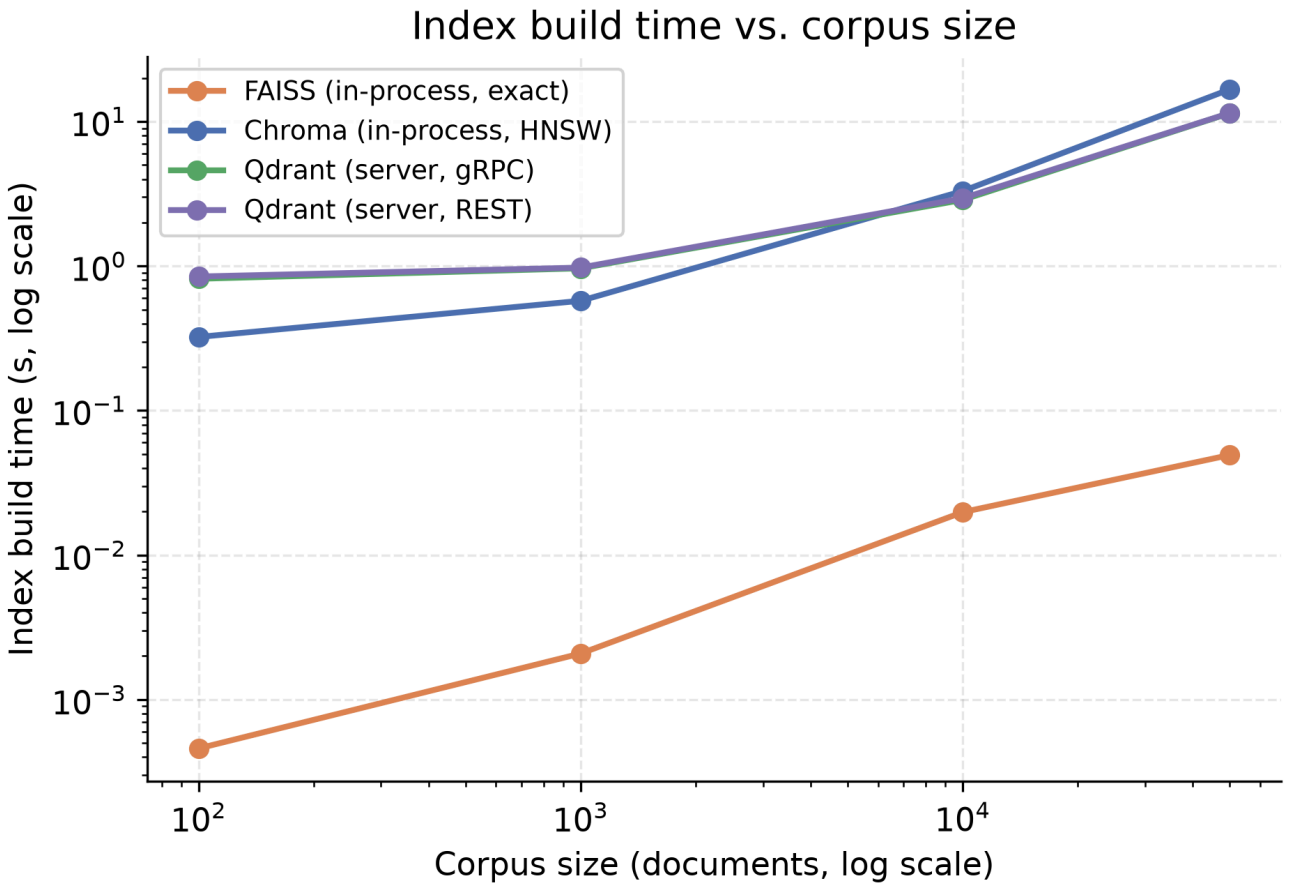
Unfiltered query latency vs. corpus size



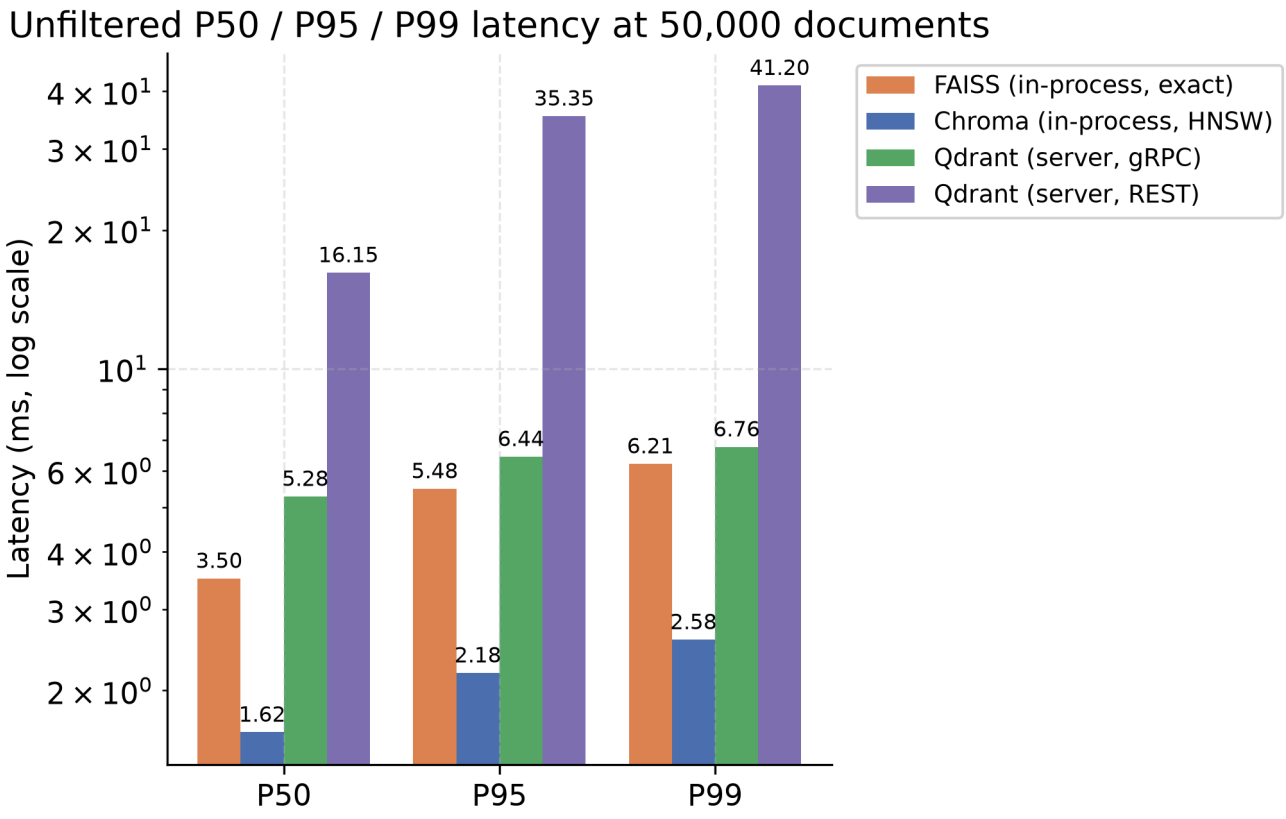
patient_id-filtered query latency vs. corpus size



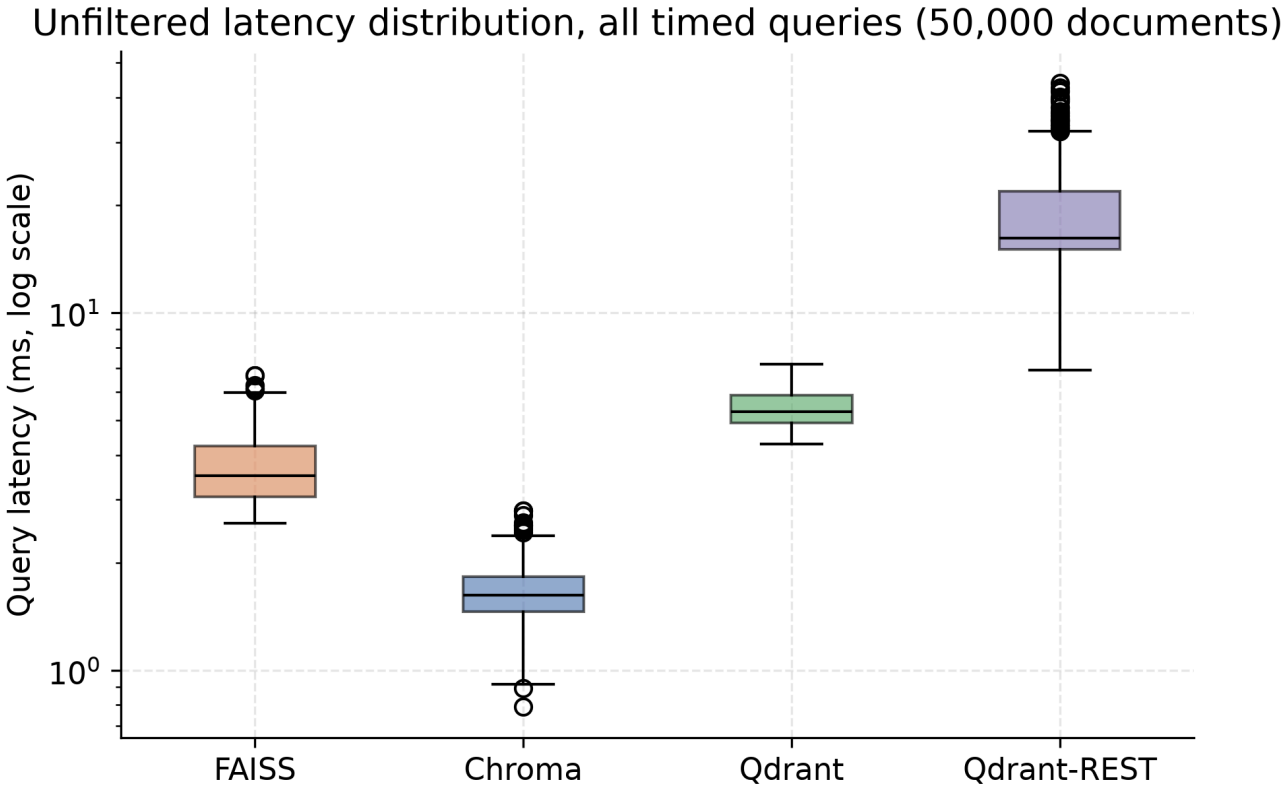
Retrieval quality with and without the filter



Index build time vs. corpus size



Latency percentiles at the largest corpus



Latency distribution at the largest corpus